

Lab 4 Report – Performance Analysis B

In this report, I aim to analyze the performance of the game state update and display sequence in the Game of Life application of Part 3. The goal is to determine the computational cost of a single game state update step, and how it is divided between I/O polling, game board drawing and game logic.

To perform the analysis, I triggered an update step starting from the initial board configuration given in the lab instructions. Breakpoints are used to measure data for the following sections:

- Total step when no input (just polling): breakpoint (start and end) at the beginning of the infinite loop. This should only include reading from the PS/2 and drawing the cursor on the VGA display.
- Total step with an update: breakpoint (start and end) at the beginning of the infinite loop.
- Polling for keyboard input for update step: two breakpoints, start at the beginning of the infinite loop and end at the switch case for “n” keyboard press (right before branching to GoL_update_board).
- Updating the board state: two breakpoints, at the start and end of GoL_update_board function (deciding which cells needs to be updated and drawing the updated board).

The results can be read from the Counters pane and are presented in Table 1.

Table 1. Data of the performance analysis for a single game state update step.

	Total step with no keyboard input	Total update step	Detecting “n” keyboard press	Update logic
# of executed instructions at start	1888390	1717489	1717489	1717515
# of executed instructions at end	1895478	1881257	1717513	1874182
# of executed instructions	7088	163768	24	156667
# of data loads at start	8376	2898	2898	2901
# of data loads at end	8522	8225	2901	8073
# of data loads	146	5327	3	5172
# of data stores at start	209868	190058	190058	190059
# of data stores at end	210734	209000	190059	208134
# of data stores	866	18942	1	18075

We may use the number of executed instructions as a proxy for time spent. From the data, we can see that a total update step (163768 executed instructions) takes significantly longer than a total step which detects no keyboard input (7088 executed instructions). This is because the total step with no keyboard input only needs to call the PS/2 subroutine and verify the read bit, and then draw the cursor on the VGA, whereas the total update step does that in addition to matching the input to a “n”, calculating the number of neighbors for each cell in the grid and drawing the cells on the grid that had a change in state. In an update step overall, there are $5327 + 18942 = 24269$ data memory accesses, which accounts for $24269/163768 = 14.8\%$ of total executed instructions. Of course, this is only for the specific initial configuration we are measuring, and this value will depend on the configuration as it is based on how many cells change state. This is a significant portion of the time spent, and this is probably mostly due to redrawing the game state and cursor on the VGA display. Also, almost all the time is spent doing actual update logic, which has 156667 executed instructions, versus polling for the “n” keyboard press, which has only 24 executed instructions.